



Fig. 6.1. Position of type checker.

its context. For example, the built-in arithmetic operator `mod` in Pascal requires integer operands, so a type checker must verify that the operands of `mod` have type integer. Similarly, the type checker must verify that dereferencing is applied only to a pointer, that indexing is done only on an array, that a user-defined function is applied to the correct number and type of arguments, and so forth. A specification of a simple type checker appears in Section 6.2. The representation of types and the question of when two types match are discussed in Section 6.3.

Type information gathered by a type checker may be needed when code is generated. For example, arithmetic operators like `+` usually apply to either integers or reals, perhaps to other types, and we have to look at the context of `+` to determine the sense that is intended. A symbol that can represent different operations in different contexts is said to be "overloaded." Overloading may be accompanied by coercion of types, where a compiler supplies an operator to convert an operand into the type expected by the context.

A distinct notion from overloading is that of "polymorphism." The body of a polymorphic function can be executed with arguments of several types. A unification algorithm for inferring types of polymorphic functions concludes this chapter.

6.1 TYPE SYSTEMS

The design of a type checker for a language is based on information about the syntactic constructs in the language, the notion of types, and the rules for assigning types to language constructs. The following excerpts from the Pascal report and the C reference manual, respectively, are examples of information that a compiler writer might have to start with.

- "If both operands of the arithmetic operators of addition, subtraction and multiplication are of type integer, then the result is of type integer."
- "The result of the unary `&` operator is a pointer to the object referred to by the operand. If the type of the operand is '`...`', the type of the result is 'pointer to `...`'."

Implicit in the above excerpts is the idea that each expression has a type

associated with it. Furthermore, types have structure; the type "pointer to ..." is constructed from the type that "..." refers to.

In both Pascal and C, types are either basic or constructed. Basic types are the atomic types with no internal structure as far as the programmer is concerned. In Pascal, the basic types are boolean, character, integer, and real. Subrange types, like 1..10, and enumerated types, like

(violet, indigo, blue, green, yellow, orange, red)

can be treated as basic types. Pascal allows a programmer to construct types from basic types and other constructed types, with arrays, records, and sets being examples. In addition, pointers and functions can also be treated as constructed types.

Type Expressions

The type of a language construct will be denoted by a "type expression." Informally, a type expression is either a basic type or is formed by applying an operator called a *type constructor* to other type expressions. The sets of basic types and constructors depend on the language to be checked.

This chapter uses the following definition of *type expressions*:

1. A basic type is a type expression. Among the basic types are *boolean*, *char*, *integer*, and *real*. A special basic type, *type_error*, will signal an error during type checking. Finally, a basic type *void* denoting "the absence of a value" allows statements to be checked.
2. Since type expressions may be named, a type name is a type expression. An example of the use of type names appears in 3(c) below; type expressions containing names are discussed in Section 6.3.
3. A type constructor applied to type expressions is a type expression. Constructors include:
 - a) *Arrays*. If T is a type expression, then $\text{array}(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I . I is often a range of integers. For example, the Pascal declaration

`var A: array[1..10] of integer;`

associates the type expression $\text{array}(1..10, \text{integer})$ with A .

- b) *Products*. If T_1 and T_2 are type expressions, then their Cartesian product $T_1 \times T_2$ is a type expression. We assume that $\times$ associates to the left.
 - c) *Records*. The difference between a record and a product is that the fields of a record have names. The *record* type constructor will be applied to a tuple formed from field names and field types. (Technically, the field names should be part of the type constructor, but it is

convenient to keep field names together with their associated types. In Chapter 8, the type constructor *record* is applied to a pointer to a symbol table containing entries for the field names.) For example, the Pascal program fragment

```
type row = record
    address: integer;
    lexeme: array [1..15] of char
end;
var table: array [1..101] of row;
```

declares the type name *row* representing the type expression

$record((address \times integer) \times (lexeme \times array(1..15, char)))$

and the variable *table* to be an array of records of this type.

- d) *Pointers*. If T is a type expression, then $pointer(T)$ is a type expression denoting the type "pointer to an object of type T ." For example, in Pascal, the declaration

```
var p: ↑ row
```

declares variable *p* to have type $pointer(row)$.

- e) *Functions*. Mathematically, a function maps elements of one set, the *domain*, to another set, the *range*. We may treat functions in programming languages as mapping a *domain type* D to a *range type* R . The type of such a function will be denoted by the type expression $D \rightarrow R$. For example, the built-in function *mod* of Pascal has domain type $int \times int$, i.e., a pair of integers, and range type int . Thus, we say *mod* has the type¹

$int \times int \rightarrow int$

As another example, the Pascal declaration

```
function f(a, b: char) : ↑ integer; ...
```

says that the domain type of *f* is denoted by $char \times char$ and range type by $pointer(integer)$. The type of *f* is thus denoted by the type expression

$char \times char \rightarrow pointer(integer)$

Often, for implementation reasons discussed in the next chapter, there are limitations on the type that a function may return; e.g., no arrays or functions may be returned. However, there are languages, of which Lisp is the most prominent example, that allow functions to

¹ We assume that $\times$ has higher precedence than $\rightarrow$, so $int \times int \rightarrow int$ is the same as $(int \times int) \rightarrow int$. Also, $\rightarrow$ associates to the right.

$$\begin{array}{ll}
 P \rightarrow D ; E & \\
 D \rightarrow D ; D & \\
 D \rightarrow \text{id} : T & \{ \text{addtype}(\text{id.entry}, T.\text{type}) \} \\
 T \rightarrow \text{char} & \{ T.\text{type} := \text{char} \} \\
 T \rightarrow \text{integer} & \{ T.\text{type} := \text{integer} \} \\
 T \rightarrow \uparrow T_1 & \{ T.\text{type} := \text{pointer}(T_1.\text{type}) \} \\
 T \rightarrow \text{array} [\text{num}] \text{ of } T_1 & \{ T.\text{type} := \text{array}(1..\text{num.val}, T_1.\text{type}) \}
 \end{array}$$

Fig. 6.4. The part of a translation scheme that saves the type of an identifier.

Type Checking of Expressions

In the following rules, the synthesized attribute *type* for *E* gives the type expression assigned by the type system to the expression generated by *E*. The following semantic rules say that constants represented by the tokens *literal* and *num* have type *char* and *integer*, respectively:

$$\begin{array}{ll}
 E \rightarrow \text{literal} & \{ E.\text{type} := \text{char} \} \\
 E \rightarrow \text{num} & \{ E.\text{type} := \text{integer} \}
 \end{array}$$

We use a function *lookup(e)* to fetch the type saved in the symbol-table entry pointed to by *e*. When an identifier appears in an expression, its declared type is fetched and assigned to the attribute *type*:

$$E \rightarrow \text{id} \quad \{ E.\text{type} := \text{lookup}(\text{id.entry}) \}$$

The expression formed by applying the mod operator to two subexpressions of type *integer* has type *integer*; otherwise, its type is *type_error*. The rule is

$$E \rightarrow E_1 \text{ mod } E_2 \quad \{ E.\text{type} := \text{if } E_1.\text{type} = \text{integer and } E_2.\text{type} = \text{integer then integer else type_error} \}$$

In an array reference $E_1[E_2]$, the index expression E_2 must have type *integer*, in which case the result is the element type *t* obtained from the type $\text{array}(s, t)$ of E_1 ; we make no use of the index set *s* of the array.

$$E \rightarrow E_1 [E_2] \quad \{ E.\text{type} := \text{if } E_2.\text{type} = \text{integer and } E_1.\text{type} = \text{array}(s, t) \text{ then } t \text{ else type_error} \}$$

Within expressions, the postfix operator $\uparrow$ yields the object pointed to by its operand. The type of $E \uparrow$ is the type *t* of the object pointed to by the pointer *E*:

$$E \rightarrow E_1 \uparrow \quad \{ E.\text{type} := \text{if } E_1.\text{type} = \text{pointer}(t) \text{ then } t \text{ else type_error} \}$$

We leave it to the reader to add productions and semantic rules to permit

additional types and operations within expressions. For example, to allow identifiers to have the type *boolean*, we can introduce the production $T \rightarrow \text{boolean}$ to the grammar of Fig. 6.3. The introduction of comparison operators like $<$ and logical connectives like *and* into the productions for E would allow the construction of expressions of type *boolean*.

Type Checking of Statements

Since language constructs like statements typically do not have values, the special basic type *void* can be assigned to them. If an error is detected within a statement, the type assigned to the statement is *type_error*.

The statements we consider are assignment, conditional, and while statements. Sequences of statements are separated by semicolons. The productions in Fig. 6.5 can be combined with those of Fig. 6.3 if we change the production for a complete program to $P \rightarrow D ; S$. A program now consists of declarations followed by statements; the above rules for checking expressions are still needed because statements can have expressions within them.

$S \rightarrow \text{Id} := E$	{ $S.\text{type} :=$ if $\text{Id.type} = E.\text{type}$ then <i>void</i> else <i>type_error</i> }
$S \rightarrow \text{if } E \text{ then } S_1$	{ $S.\text{type} :=$ if $E.\text{type} = \text{boolean}$ then $S_1.\text{type}$ else <i>type_error</i> }
$S \rightarrow \text{while } E \text{ do } S_1$	{ $S.\text{type} :=$ if $E.\text{type} = \text{boolean}$ then $S_1.\text{type}$ else <i>type_error</i> }
$S \rightarrow S_1 ; S_2$	{ $S.\text{type} :=$ if $S_1.\text{type} = \text{void}$ and $S_2.\text{type} = \text{void}$ then <i>void</i> else <i>type_error</i> }

Fig. 6.5. Translation scheme for checking the type of statements.

Rules for checking statements are given in Fig. 6.5. The first rule checks that the left and right sides of an assignment statement have the same type.³ The second and third rules specify that expressions in conditional and while statements must have type *boolean*. Errors are propagated by the last rule in Fig. 6.5 because a sequence of statements has type *void* only if each sub-statement has type *void*. In these rules, a mismatch of types produces the type *type_error*; a friendly type checker would, of course, report the nature and location of the type mismatch as well.

³ If an expression is allowed on the left side of an assignment, then we also have to distinguish between *l*-values and *r*-values. For example, $1 := 2$ is incorrect because the constant 1 cannot be assigned to.

Type Checking of Functions

The application of a function to an argument can be captured by the production

$$E \rightarrow E (E)$$

in which an expression is the application of one expression to another. The rules for associating type expressions with nonterminal T can be augmented by the following production and action to permit function types in declarations.

$$T \rightarrow T_1 \rightarrow T_2 \quad \{ T.type := T_1.type \rightarrow T_2.type \}$$

Quotes around the arrow used as a function constructor distinguish it from the arrow used as the metasymbol in a production.

The rule for checking the type of a function application is

$$E \rightarrow E_1 (E_2) \quad \{ E.type := \text{if } E_2.type = s \text{ and } E_1.type = s \rightarrow t \text{ then } t \text{ else } type_error \}$$

This rule says that in an expression formed by applying E_1 to E_2 , the type of E_1 must be a function $s \rightarrow t$ from the type s of E_2 to some range type t ; the type of $E_1 (E_2)$ is t .

Many issues related to type checking in the presence of functions can be discussed with respect to the simple syntax above. The generalization to functions with more than one argument is done by constructing a product type consisting of the arguments. Note that n arguments of type $T_1, \dots, T_n$ can be viewed as a single argument of type $T_1 \times \dots \times T_n$. For example, we might write

$$root : (real \rightarrow real) \times real \rightarrow real \quad (6.1)$$

to declare a function `root` that takes a function from reals to reals and a real as arguments and returns a real. Pascal-like syntax for this declaration is

```
function root (function f (real): real; x: real): real
```

The syntax in (6.1) separates the declaration of the type of a function from the names of its parameters.

6.3 EQUIVALENCE OF TYPE EXPRESSIONS

The checking rules in the last section have the form, "if two type expressions are equal then return a certain type else return *type_error*." It is therefore important to have a precise definition of when two type expressions are equivalent. Potential ambiguities arise when names are given to type expressions and the names are then used in subsequent type expressions. The key issue is whether a name in a type expression stands for itself or whether it is an abbreviation for another type expression.

Since there is interaction between the notion of equivalence of types and the